



ESSDK内存拷贝用户手册

Rev 1.1

2026-05-14

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

版本号	日期	说明
v1.1	2026.05.14	修改概述格式错误
v1.0	2026.05.11	重构文档结构，增加 python 接口
v0.1	2024.11.04	初始版本

目录

声明 1

 知识产权声明 1

 责任限制 1

修订记录 2

1. 概述 4

 1.1. 核心机制：CMDQ 与 Workqueue 4

 1.2. 主要功能与优势 4

 1.3. 用户态接口 4

2. C API 接口说明 5

 2.1. API 详细说明 5

 2.1.1. ES_CmdQueue_Create 5

 2.1.2. ES_CmdQueue_Destroy 5

 2.1.3. ES_CmdQueue_Sync 6

 2.1.4. ES_CmdQueue_Query 6

 2.1.5. ES_Memcpy_Async 7

 2.1.6. ES_Memcpy_Async_Wait_Notifier 8

 2.2. 数据类型 8

 2.2.1. esw_cmdq_query 8

 2.2.2. esw_memcp_f2f_cmd 9

3. Python API 接口说明 10

 3.1. API 详细说明 10

 3.1.1. command_queue_create() 10

 3.1.2. command_queue_destroy() 10

 3.1.3. command_queue_sync() 11

 3.1.4. command_queue_query() 12

 3.1.5. memory_copy_async() 13

 3.1.6. memory_copy_async_wait_notifier() 14

 3.2. 数据类型 15

 3.2.1. CmdqQuery 15

 3.2.2. esw_memcp_f2f_cmd 16

 3.3. 使用示例 17

 3.3.1. 基本内存复制 17

 3.3.2. 多任务批量传输 18

 3.3.3. 等待通知机制 19

4. 错误码 21

1. 概述

MemCP (Memory Copy) 模块是 ESSDK 提供的高效内存传输与任务调度子系统。它通过统一的用户态接口 (`libes_memcp.so`)，向上层应用屏蔽底层硬件细节，实现高效的异步数据传输与命令队列管理。模块内核层深度融合了**命令队列 (CMDQ)**与**内存复制 (Memcpy)**功能，为复杂的内存操作和多任务并发场景提供了一致性强、易用性高的解决方案。

1.1. 核心机制：CMDQ 与 Workqueue

CMDQ 功能在内核层基于 **Linux Workqueue** 机制实现，专注于高并发、低延迟的异步任务调度：

- **独立队列管理**：通过 `open` 系统调用创建 CMDQ 时，内核会返回一个唯一的文件描述符 (`fd`)，并与一个独立的工作队列绑定。支持创建多个 CMDQ 实例，实现多队列隔离与并行调度。
- **智能资源调度**：CMDQ 允许用户为多个外设创建 DMA 任务队列，内核根据负载动态分配 DMA 通道，确保数据搬运任务的高效完成。
- **自动化资源回收**：队列销毁时 (`close`)，相关内核资源自动释放，避免资源泄漏。

1.2. 主要功能与优势

MemCP 模块通过封装 CMDQ 与 Memicpy，提供以下核心能力：

1. **异步任务调度** 基于内核 Workqueue 实现，用户仅需将任务下发至队列即可返回，无需阻塞等待。内核负责按序或按优先级执行任务，显著提升系统吞吐率。
2. **高效内存复制** 支持通过 CMDQ 下发内存复制任务，底层利用 DMA 通道进行数据传输，极大降低 CPU 占用率，确保大数据量搬运的高效性与可靠性。
3. **精细化同步与查询**
 - **同步等待**：提供接口确保队列中所有命令执行完毕后才返回，满足强一致性需求。
 - **状态查询**：支持实时查询队列状态（如任务进度、队列空满），便于上层应用进行流控与异常处理。
4. **完善的错误处理** 提供详细的错误码与内核日志，帮助用户快速定位 DMA 传输、队列溢出或参数配置等问题。

1.3. 用户态接口

系统通过 `libes_memcp.so` 动态库对外暴露 API。应用程序只需链接该库，即可便捷地调用 MemCP 的创建、提交、同步等接口，实现高效的内存管理与命令执行。

2. C API 接口说明

2.1. API 详细说明

2.1.1. ES_CmdQueue_Create

【函数体】

```
ES_S32 ES_CmdQueue_Create(ES_S32 *cmdQHandle)
```

【描述】

创建并初始化一个命令队列，返回队列句柄。

- 1. 调用该接口将创建一个命令队列，并返回一个文件描述符 cmdQHandle，该文件描述符是命令队列的唯一标识。
- 2. 创建的文件描述符与内核中的工作队列（workqueue）绑定，用于异步处理命令队列中的任务。
- 3. 用户通过 cmdQHandle 对队列进行后续操作（如添加任务、同步、销毁等）。
- 4. 每次调用 ES_CmdQueue_Create，都会创建一个新的命令队列和对应的工作队列。

【参数】

参数名称	描述	输入/输出
cmdQHandle	返回的命令队列句柄，用于后续操作	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.1.2. ES_CmdQueue_Destroy

【函数体】

```
ES_S32 ES_CmdQueue_Destroy(ES_S32 cmdQHandle)
```

【描述】

销毁命令队列，释放相关资源。

- 1. 通过传入的命令队列句柄 cmdQHandle，销毁与该句柄绑定的工作队列及其他资源。
- 2. 调用此接口实际会调用系统的 close 函数，确保文件描述符关闭并释放其关联的内核资源。
- 3. 在 close 操作完成后，队列及其任务将不再可用，相关资源将被回收。
- 4. 确保在销毁队列前已完成所有任务或通过 ES_CmdQueue_Sync 确认任务完成，避免资源泄露或未完成任务的丢失。

【参数】

参数名称	描述	输入/输出
cmdQHandle	要销毁的命令队列句柄	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

在使用完命令队列后应调用 ES_CmdQueue_Destroy 销毁，以释放资源。

2.1.3. ES_CmdQueue_Sync

【函数体】

```
ES_S32 ES_CmdQueue_Sync(ES_S32 cmdQHandle)
```

【描述】

阻塞调用线程，直到命令队列中的所有任务执行完成。

- 1. 通过命令队列句柄 cmdQHandle 确定需要同步的队列。
- 2. 调用该接口后，调用线程将阻塞，直到队列中所有任务完成。
- 3. 常用于确保所有任务在继续后续操作之前已处理完毕。

【参数】

参数名称	描述	输入/输出
cmdQHandle	要同步的命令队列句柄	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.1.4. ES_CmdQueue_Query

【函数体】

```
ES_S32 ES_CmdQueue_Query(ES_S32 cmdQHandle, struct esw_cmdq_query *query)
```

【描述】

查询命令队列的状态。

- 1. 返回命令队列当前状态，包括是否空闲（status 0 代表空闲 1 代表忙碌）以及剩余未完成任务的数量（task_count）。
- 2. 用户可通过该接口动态监控任务进度，用于实现任务调度的智能决策。

【参数】

参数名称	描述	输入/输出
cmdQHandle	要查询的命令队列句柄	输入

参数名称	描述	输入/输出
query	用于存储查询结构的结构体	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.1.5. ES_Memcpy_Async

【函数体】

```
ES_S32 ES_Memcpy_Async(  
    ES_S32 cmdQHandle,  
    ES_U64 srcFd,  
    ES_U32 srcOffset,  
    ES_U64 dstFd,  
    ES_U32 dstOffset,  
    ES_U64 len,  
    ES_S32 timeout)
```

【描述】

发起一个异步内存复制任务，并将任务添加到指定命令队列中。

- 1. 用户需提供源地址和目标地址的文件描述符（fd）、偏移量、复制长度和超时时间。
- 2. 复制任务将在后台异步执行，不会阻塞调用线程。
- 3. 内存复制操作通过 DMA 完成，系统自动分配 DMA 通道以实现高效传输。

【参数】

参数名称	描述	输入/输出
cmdQHandle	目标命令队列的句柄，用于分配任务	输入
srcFd	源文件描述符，表示源内存的 DMA-BUF 句柄	输入
srcOffset	源内存偏移量，单位为字节	输入
dstFd	目标文件描述符，表示目标内存的 DMA-BUF 句柄	输入
dstOffset	目标内存偏移量，单位为字节	输入
len	要复制的数据长度，以字节为单位	输入
timeout	内存复制操作超时时间，以毫秒为单位	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

- 文件描述符（fd）是命令队列的核心标识符，任何操作都必须依赖于有效的 cmdQHandle。
- 严格遵循队列的创建与销毁流程，未及时销毁的队列可能会导致内核资源泄露。
- 每个队列 cmdQHandle 独立，支持多个命令队列并发使用。

2.1.6. ES_Memcpy_Async_Wait_Notifier

【函数体】

ES_S32 ES_Memcpy_Async_Wait_Notifier(ES_S32 cmdQHandle)

【描述】

在异步内存拷贝任务提交后，等待并检测任务执行状态，专门用于捕获 DMA 操作超时（timeout）时内核返回的错误。

1. 监测有效的 cmdQHandle 描述符上的拷贝任务。
2. 替代 ES_CmdQueue_Sync 同步操作；
3. 当 ES_Memcpy_Async 设置的超时时间（timeout 参数）触发后，内核无法通过 ioctl 直接返回错误，通过等待内核事件通知获取实际错误，补充 ioctl 异步提交的错误捕获能力。

【参数】

参数名称	描述	输入/输出
cmdQHandle	目标命令队列的句柄	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

- 必须紧接在 ES_Memcpy_Async 后调用，针对同一队列中的最近提交任务。
- 使用此接口不需要再调用 ES_CmdQueue_Sync。

2.2. 数据类型

相关数据类型、数据结构定义如下：

2.2.1. esw_cmdq_query

【说明】

定义命令队列的查询结果结构体，用于返回当前命令队列的状态信息。

【定义】

```
struct esw_cmdq_query {
    int status;
```

```
int task_count;
int last_error;
};
```

【成员】

成员	描述
status	队列状态，0 表示空闲（FREE），1 表示繁忙（BUSY）。
task_count	队列中当前未完成的任务数量。
last_error	最后一个失败任务的错误码。

2.2.2. esw_memcp_f2f_cmd

【说明】

定义了一个结构体，用于表示从源缓冲区到目标缓冲区的内存拷贝命令，包括源和目标的文件描述符、偏移量、数据长度以及操作超时时间。

【定义】

```
struct esw_memcp_f2f_cmd {
    int src_fd;
    unsigned int src_offset;
    int dst_fd;
    unsigned int dst_offset;
    size_t len;
    int timeout;
};
```

【成员】

成员	描述
src_fd	源缓冲区的文件描述符。
src_offset	源缓冲区中数据拷贝开始的偏移量。
dst_fd	目标缓冲区的文件描述符。
dst_offset	目标缓冲区中数据将被拷贝到的偏移量。
len	要拷贝的数据长度，单位为字节
timeout	内存拷贝操作的超时时间，单位为毫秒。

3. Python API 接口说明

3.1. API 详细说明

3.1.1. command_queue_create()

【函数原型】

```
def command_queue_create() -> int
```

【功能描述】

创建一个新的命令队列，用于管理异步 DMA 内存复制任务。系统返回命令队列句柄 `cmdQHandle` 作为唯一标识，并在内核层创建对应的工作队列。

【参数】

无

【返回值】

类型	说明
int	成功返回命令队列句柄（非负整数），失败抛出异常

【异常】

异常类型	触发条件
ESMemcpError	队列创建失败，包含具体错误码

提示

- 每次调用创建独立的命令队列
- 返回的句柄用于后续所有队列操作
- 必须调用 `command_queue_destroy()` 释放资源

【示例】

```
from es_py import memcp

# 创建命令队列
cmd_q_handle = memcp.command_queue_create()
print(f"队列创建成功，句柄: {cmd_q_handle}")
```

3.1.2. command_queue_destroy()

【函数原型】

```
def command_queue_destroy(cmdQHandle: int) -> int
```

【功能描述】

销毁指定的命令队列，关闭文件描述符并释放所有关联的内核资源。

【参数】

参数名	类型	方向	说明
cmdQHandle	int	输入	命令队列句柄（由 command_queue_create() 返回）

【返回值】

类型	说明
int	成功返回 0，失败抛出异常

【异常】

异常类型	触发条件
ESMemcpError	队列销毁失败

提示

- 销毁前建议先调用 command_queue_sync() 确保任务完成
- 销毁后句柄立即失效，不可再使用
- 未完成任务将被取消

【示例】

```
from es_py import memcp

cmd_q_handle = memcp.command_queue_create()
# 使用队列...
memcp.command_queue_destroy(cmd_q_handle)
print("队列已销毁")
```

3.1.3. command_queue_sync()

【函数原型】

```
def command_queue_sync(cmdQHandle: int) -> int
```

【功能描述】

阻塞等待指定命令队列中的所有任务执行完成。调用线程将被挂起，直到队列变为空闲状态。

【参数】

参数名	类型	方向	说明
cmdQHandle	int	输入	命令队列句柄

【返回值】

类型	说明
int	成功返回 0，失败抛出异常

【异常】

异常类型	触发条件
ESMemcpError	同步操作失败

提示

- 阻塞调用，适合需要等待所有任务完成的场景
- 队列空闲时立即返回
- 可被用于多任务批处理后的统一等待

【示例】

```
from es_py import memcp

# 提交多个任务
for i in range(10):
    memcp.memory_copy_async(cmd_q_handle, ...)

# 等待所有任务完成
memcp.command_queue_sync(cmd_q_handle)
print("所有任务已完成")
```

3.1.4. command_queue_query()

【函数原型】

```
def command_queue_query(cmdQHandle: int) -> CmdqQuery
```

【功能描述】

查询指定命令队列的当前状态，包括队列忙/闲状态、未完成任务数量和最后错误码。

【参数】

参数名	类型	方向	说明
cmdQHandle	int	输入	命令队列句柄

【返回值】

类型	说明
CmdqQuery	查询结果结构体，包含队列状态信息

【异常】

异常类型	触发条件
ESMemcpError	查询操作失败

提示

- 非阻塞调用，立即返回当前状态
- 可用于轮询监控任务进度
- status 为 0 表示空闲，1 表示忙碌

【示例】

```
from es_py import memcp

query = memcp.command_queue_query(cmd_q_handle)
print(f"队列状态: {'忙碌' if query.status else '空闲'}")
print(f"待完成任务: {query.task_count}")
if query.last_error:
    print(f"最后错误: 0x{query.last_error:X}")
```

3.1.5. memory_copy_async()

【函数原型】

```
def memory_copy_async(
    cmdQHandle: int,
    srcFd: int,
    srcOffset: int,
    dstFd: int,
    dstOffset: int,
    len: int,
    timeout: int
) -> int
```

【功能描述】

提交异步内存复制任务到指定命令队列。任务通过 DMA 硬件引擎在后台执行，调用立即返回不阻塞。

【参数】

参数名	类型	方向	取值范围	说明
cmdQHandle	int	输入	> 0	目标命令队列句柄
srcFd	int	输入	>= 0	源内存文件描述符 (DMA-BUF 句柄)
srcOffset	int	输入	0 ~ 4GB	源内存起始偏移量 (字节)
dstFd	int	输入	>= 0	目标内存文件描述符 (DMA-BUF 句柄)
dstOffset	int	输入	0 ~ 4GB	目标内存起始偏移量 (字节)
len	int	输入	> 0	复制数据长度 (字节)
timeout	int	输入	> 0	DMA 传输超时时间 (毫秒)

【返回值】

类型	说明
int	成功返回 0（任务已加入队列），失败抛出异常

【异常】

异常类型	触发条件
ESMemcpError	任务提交失败

提示

- 内存文件描述符须通过 es_py.memory 模块分配
- 对于 CACHED 模式内存，传输前必须调用 memory_flush_cache() 刷新缓存
- 偏移量和长度必须满足 DMA 对齐要求
- 超时时间建议设置为 5000ms 以上

【示例】

```
from es_py import memcp, memory, common

# 分配内存
fd_src = memory.memory_alloc(common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED,
                              "src", "system", 1024*1024)
fd_dst = memory.memory_alloc(common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED,
                              "dst", "system", 1024*1024)

# 提交 DMA 传输
memcp.memory_copy_async(cmd_q_handle, fd_src, 0, fd_dst, 0, 1024*1024, 5000)
```

3.1.6. memory_copy_async_wait_notifier()

【函数原型】

```
def memory_copy_async_wait_notifier(cmdQHandle: int) -> int
```

【功能描述】

等待异步内存复制任务完成通知。该接口专用于捕获 DMA 传输超时等异步执行过程中的错误，弥补 memory_copy_async() 无法直接返回超时错误的限制。

【参数】

参数名	类型	方向	说明
cmdQHandle	int	输入	命令队列句柄

【返回值】

类型	说明
int	成功返回 0，失败抛出异常

【异常】

异常类型	触发条件
ESMemcpError	DMA 任务失败（含超时）

提示

- 必须紧接在 memory_copy_async() 后调用
- 仅针对最近一次提交的任务
- 使用此接口后无需再调用 command_queue_sync()
- 适合需要精确捕获超时错误的场景

【示例】

```
from es_py import memcp

try:
    # 提交异步传输
    memcp.memory_copy_async(cmd_q_handle, fd_src, 0, fd_dst, 0, size, 5000)

    # 等待完成通知
    memcp.memory_copy_async_wait_notifier(cmd_q_handle)
    print("传输成功完成")
except Exception as e:
    if "TIMEOUT" in str(e):
        print("DMA 传输超时")
    else:
        print(f"传输失败: {e}")
```

3.2. 数据类型

3.2.1. CmdqQuery

【结构说明】

命令队列状态查询结果结构体，用于返回队列的运行状态信息。

【类定义】

```
class CmdqQuery:
    status: int # 队列状态
    task_count: int # 未完成任务数
    last_error: int # 最后错误码
```

【成员说明】

成员名	类型	说明
status	int	队列状态：0=空闲（FREE），1=忙碌（BUSY）

成员名	类型	说明
task_count	int	队列中当前未完成任务数量
last_error	int	最近一次失败任务的错误码（0 表示无错误）

【示例】

```
from es_py import memcp

# 查询队列状态
query = memcp.command_queue_query(cmd_q_handle)

# 检查队列是否繁忙
if query.status == 1:
    print(f"队列繁忙，剩余 {query.task_count} 个任务")
else:
    print("队列空闲")

# 检查是否有错误
if query.last_error != 0:
    print(f"检测到错误，错误码: 0x{query.last_error:08X}")
```

3.2.2. esw_memcp_f2f_cmd

【结构说明】

内存复制命令参数结构体（内部使用）。该结构体描述了从源缓冲区到目标缓冲区的内存复制命令参数，主要在内核层使用。当前 Python 接口未直接导出该结构体，应用程序通常不需要直接操作。

【类定义】

```
class esw_memcp_f2f_cmd: # C 层内部结构（未直接导出到 Python）
    src_fd: int          # 源缓冲区文件描述符
    src_offset: int       # 源缓冲区偏移量
    dst_fd: int           # 目标缓冲区文件描述符
    dst_offset: int       # 目标缓冲区偏移量
    len: int              # 复制数据长度
    timeout: int          # 超时时间
```

【成员说明】

成员名	类型	说明
src_fd	int	源缓冲区的文件描述符（memFd）
src_offset	int	源缓冲区数据起始偏移量（字节）
dst_fd	int	目标缓冲区的文件描述符（memFd）
dst_offset	int	目标缓冲区数据写入偏移量（字节）
len	int	复制数据长度（字节）
timeout	int	DMA 传输超时时间（毫秒）

提示

该结构体为内部结构，应用程序通过 `memory_copy_async()` 函数的参数传递这些值，无需直接构造此结构体。

3.3. 使用示例

本章提供完整的代码示例，展示 MemCP 模块在实际应用中的使用方法。

3.3.1. 基本内存复制

演示最基本的 DMA 内存复制流程，包括内存分配、队列创建、异步传输和资源清理。

```
from es_py import memcp, memory, common

def basic_memory_copy():
    """基本内存复制示例"""
    mem_len = 2 * 1024 * 1024 # 2MB
    cache_mode = common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED

    fd_src = None
    fd_dst = None
    cmd_q_handle = None

    try:
        # 1. 分配源和目标内存
        fd_src = memory.memory_alloc(cache_mode, "src", "system", mem_len)
        fd_dst = memory.memory_alloc(cache_mode, "dst", "system", mem_len)

        # 2. 刷新源缓存 (CACHED 模式必须)
        memory.memory_flush_cache(fd_src)

        # 3. 创建命令队列
        cmd_q_handle = memcp.command_queue_create()

        # 4. 提交异步内存复制任务
        memcp.memory_copy_async(cmd_q_handle, fd_src, 0, fd_dst, 0,
                                mem_len, 5000)

        # 5. 等待传输完成
        memcp.command_queue_sync(cmd_q_handle)

        # 6. 刷新目标缓存 (读取前必须)
        memory.memory_flush_cache(fd_dst)

        # 7. 查询状态
        query = memcp.command_queue_query(cmd_q_handle)
        if query.last_error == 0:
            print("传输成功完成")

        return True

    except Exception as e:
        print(f"内存复制失败: {e}")
        return False

    finally:
        # 8. 清理资源
        if cmd_q_handle:
            memcp.command_queue_destroy(cmd_q_handle)
```

```

if fd_src:
    memory.memory_free(fd_src)
if fd_dst:
    memory.memory_free(fd_dst)

if __name__ == "__main__":
    basic_memory_copy()

```

提示

- CACHED 模式内存必须在 DMA 传输前后刷新缓存
- 使用 finally 块确保资源正确释放
- 查询队列状态确认传输成功

3.3.2. 多任务批量传输

演示如何使用单个命令队列批量提交多个异步传输任务。

```

from es_py import memcp, memory, common

def async_multiple_transfers():
    """多任务异步传输示例"""
    cache_mode = common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED
    block_size = 512 * 1024 # 512KB per block
    num_blocks = 4

    src_fds = []
    dst_fds = []
    cmd_q_handle = None

    try:
        # 1. 分配多组内存
        for i in range(num_blocks):
            fd_src = memory.memory_alloc(cache_mode, f"src_{i}", "system", block_size)
            fd_dst = memory.memory_alloc(cache_mode, f"dst_{i}", "system", block_size)
            src_fds.append(fd_src)
            dst_fds.append(fd_dst)

        # 2. 创建命令队列
        cmd_q_handle = memcp.command_queue_create()

        # 3. 批量提交异步传输任务
        for fd_src, fd_dst in zip(src_fds, dst_fds):
            memory.memory_flush_cache(fd_src)
            memcp.memory_copy_async(cmd_q_handle, fd_src, 0, fd_dst, 0,
                                    block_size, 5000)

        # 4. 等待所有传输完成
        memcp.command_queue_sync(cmd_q_handle)

        # 5. 刷新所有目标缓存
        for fd_dst in dst_fds:
            memory.memory_flush_cache(fd_dst)

        print(f"成功传输 {num_blocks} 个块")
        return True

```

```

except Exception as e:
    print(f"批量传输失败: {e}")
    return False

finally:
    if cmd_q_handle:
        memcp.command_queue_destroy(cmd_q_handle)
    for fd in src_fds:
        memory.memory_free(fd)
    for fd in dst_fds:
        memory.memory_free(fd)

if __name__ == "__main__":
    async_multiple_transfers()

```

提示

- 单个队列管理多个任务，减少系统开销
- DMA 硬件自动调度，提高传输效率
- 批量操作适合大规模数据处理场景

3.3.3. 等待通知机制

演示使用 `memory_copy_async_wait_notifier()` 进行事件驱动的异步等待。

```

from es_py import memcp, memory, common

def async_transfer_with_notifier():
    """等待通知机制示例"""
    mem_len = 1024 * 1024 # 1MB
    cache_mode = common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED

    fd_src = None
    fd_dst = None
    cmd_q_handle = None

    try:
        # 1. 分配内存
        fd_src = memory.memory_alloc(cache_mode, "src", "system", mem_len)
        fd_dst = memory.memory_alloc(cache_mode, "dst", "system", mem_len)

        # 2. 创建命令队列
        cmd_q_handle = memcp.command_queue_create()

        # 3. 刷新源缓存并提交传输
        memory.memory_flush_cache(fd_src)
        memcp.memory_copy_async(cmd_q_handle, fd_src, 0, fd_dst, 0,
                                mem_len, 5000)

        # 4. 使用等待通知机制（阻塞直到传输完成）
        memcp.memory_copy_async_wait_notifier(cmd_q_handle)

        # 5. 刷新目标缓存
        memory.memory_flush_cache(fd_dst)

    print("传输完成")
    return True

```

```
except Exception as e:
    print(f"异步传输失败: {e}")
    if "TIMEOUT" in str(e):
        print("建议: 增加 timeout 参数")
    return False

finally:
    if cmd_q_handle:
        memcp.command_queue_destroy(cmd_q_handle)
    if fd_src:
        memory.memory_free(fd_src)
    if fd_dst:
        memory.memory_free(fd_dst)

if __name__ == "__main__":
    async_transfer_with_notifier()
```

提示

通知机制优势:

- 事件驱动，无需主动轮询
- CPU 占用更低，适合低功耗场景
- 响应更及时，延迟更小

4. 错误码

MemCP 相关 API 错误码如下表所示。

错误码	宏定义	描述
0xA0156009	ES_MEMCP_ERROR_NULL_PTR	空指针参数错误
0xA015600A	ES_MEMCP_BUSY	命令队列繁忙
0xA015600B	ES_MEMCP_INVALID_HANDLE	命令队列句柄无效
0xA015600C	ES_MEMCP_MEMORY_ERROR	内存分配失败
0xA015600D	ES_MEMCP_OPERATION_FAIL	操作失败
0xA015600E	ES_MEMCP_UNKNOWN_CMD	未知命令
0xA015600F	ES_MEMCP_SYNC_ERROR	队列同步失败
0xA0156010	ES_MEMCP_DEV_NOT_FOUND	设备未找到
0xA0156011	ES_MEMCP_IOCTL_FAIL	IOCTL 操作失败
0xA0156012	ES_MEMCP_MEM_ALLOC_FAIL	内核内存分配失败
0xA0156013	ES_MEMCP_DMA_TIMEOUT	DMA 传输超时